MongoDB Assignment

15 Questions • Easy-to-understand answers • Approximately 80–100 words each

1. MongoDB as a Document-Oriented Database

MongoDB is a NoSQL document-oriented database. It stores information in flexible documents, usually in BSON format, inside collections. A document contains fields and values, similar to a JSON object. In a relational database, data is stored in tables with rows and columns and usually follows a fixed schema. MongoDB can store nested objects and arrays in one document, so related information can be kept together. For example, a customer document may contain the customer's name, address and phone numbers. This makes MongoDB useful for applications with changing data structures.

2. Features of Document-Oriented Databases

Document-oriented databases store data as individual documents instead of fixed relational rows. Their main features include flexible schemas, nested fields, arrays, easy handling of different data types and support for queries on document fields. MongoDB implements these features through BSON documents stored in collections. Documents in one collection do not always need identical fields. For example, two product documents can have different specifications. MongoDB also supports indexes, aggregation and CRUD operations. These features make it easier to develop applications where information changes frequently or has different structures.

3. Consistency in MongoDB

Consistency means that users should receive correct and reliable data after database operations. MongoDB provides atomicity at the single-document level, meaning an update to one document happens completely or not at all. In replica sets, MongoDB copies data from the primary server to secondary servers. Write concern lets an application choose how many members must acknowledge a write. Read concern and read preference help control the consistency and source of reads. For stronger guarantees, applications can use suitable write and read concerns, including majority acknowledgement.

4. Transactions in MongoDB

A transaction is a group of database operations treated as one unit. MongoDB supports multi-document transactions, so changes across multiple documents or collections can be committed together or rolled back if an operation fails. This follows the ACID properties: atomicity, consistency, isolation and durability. For example, when placing an order, a transaction can create the order and update stock information. If one step fails, the transaction can be aborted to avoid incomplete changes. Transactions are useful when several related updates must succeed together, though they may add performance overhead.

5. Availability and Replication in MongoDB

Availability means that a database remains accessible even when a server fails. MongoDB supports high availability through replica sets. A replica set contains one primary member and one or more secondary members. The primary normally accepts writes and records changes in an operation log. Secondary members copy these changes and maintain data copies. If the primary becomes unavailable, the remaining members can hold an election and choose a new primary. This automatic failover helps reduce downtime. Replication also provides extra copies of data, improving fault tolerance.

6. MongoDB Query Features and CRUD Operations

MongoDB supports CRUD operations: Create, Read, Update and Delete. For example, `db.students.insertOne({name: "Riya", age: 20})` creates a document. `db.students.find({age: 20})` finds matching students. `db.students.updateOne({name: "Riya"}, {$set: {age: 21}})` updates a field. `db.students.deleteOne({name: "Riya"})` deletes a matching document. Queries can filter using conditions such as `$gt` for greater than and `$in` for matching values in a list. MongoDB also supports sorting, projection, indexes and aggregation.

7. MongoDB Scaling: Vertical and Horizontal

Scaling means increasing a database's ability to handle more data and users. Vertical scaling increases the power of one server by adding CPU, memory or storage. It is simple, but the server has hardware limits. Horizontal scaling adds more servers and distributes data or workload among them. MongoDB supports horizontal scaling through sharding, where data is divided across multiple machines. It can also benefit from vertical upgrades. Vertical scaling may suit smaller systems, while horizontal scaling is useful for large applications that need more storage and processing capacity.

8. Suitable Use Cases of MongoDB

MongoDB is suitable for applications such as e-commerce websites, content management systems, mobile apps, product catalogues, event logging and analytics. These applications often store data with different fields or structures. A document database is preferred when the data model changes frequently because developers can add or remove fields without redesigning many tables. Related information can also be stored as nested documents or arrays. For example, products may have different specifications, such as size, colour or memory. MongoDB's flexible documents make these varied records easier to manage.

9. MongoDB for Event Logging

MongoDB can store application or system events as documents in an `events` collection. Each document can record what happened, when it happened and which user or service caused it. A suitable structure is: `{eventType: "login", userId: 101, timestamp: ISODate("2026-11-02T10:00:00Z"), source: "mobile-app", status: "success", details: {device: "Android"}}`. Different event types can include different details because MongoDB has a flexible schema. Indexes on `timestamp`, `eventType` and `userId` can help search and analyse events efficiently.

10. MongoDB in Content Management Systems

A Content Management System (CMS) is used to create, store and manage digital content such as articles, images and pages. MongoDB is useful for a CMS because its document model supports different content structures. An article may contain a title, author, body, images and tags, while a product page may contain specifications and prices. These fields can be stored in flexible documents. New fields can be added when content requirements change. Nested objects can also represent sections or media details, making MongoDB suitable for dynamic and varied content.

11. MongoDB in Blogging Platforms

MongoDB can manage the data of a blogging platform using collections such as `users`, `posts` and `comments`. A user document may store name, email and profile details. A post document may contain title, body, authorId, tags, createdAt and status. A comment document may contain postId, userId, text and createdAt. Tags can be stored as an array inside each post, for example `tags: ["technology", "learning"]`. References such as `authorId` and `postId` connect related records. This structure supports flexible blog content and easy updates.

12. MongoDB in Web and Real-Time Analytics

Web analytics collects information about website visits, clicks, searches and user activity. MongoDB can store these events as documents with fields such as userId, page, action and timestamp. Its flexible schema allows different event types to contain different details. Queries and indexes help filter events by date, user or action. The aggregation framework can group records and calculate counts, totals and other summaries. MongoDB can scale horizontally through sharding to handle growing data volumes and traffic. These features make it useful for dashboards, activity tracking and near-real-time analysis.

13. MongoDB in E-Commerce Applications

MongoDB can support an e-commerce application through collections such as `products`, `customers`, `orders` and `carts`. A product document can store name, price, category and different specifications. A customer document can contain contact and delivery details. An order document can store customerId, items, quantities, total amount and order status. A cart document can contain customerId and selected products. Product details can vary because different categories have different features. References connect collections, while embedded documents can keep order items together. This flexible design supports changing store requirements.

14. Challenges of Complex Transactions in MongoDB

Complex transactions can be challenging when one business activity changes many documents or collections. For example, an online order may need to reduce product stock, save payment details and create an order record. If only some operations succeed, the database may contain incomplete information. MongoDB supports multi-document transactions to group these operations into one unit. The application starts a transaction, performs the updates and commits them only if all steps succeed. If an error occurs, it can abort the transaction. Good design is still important because long transactions can affect performance.

15. Queries on Varying Aggregate Structures

MongoDB can query documents even when records in the same collection have different fields or nested structures. For example, one product document may contain `size`, while another contains `storage` and `colour`. MongoDB can filter documents using fields that exist in each record, and it can query nested values and array elements. Its flexible schema allows applications to add new fields without changing every existing document. This is useful when requirements change or data comes from different sources. It supports faster development, but consistent field naming and validation remain important.